TP 2 : Méthodes de segmentation basées histogramme

L'objectif de ce TP est d'utiliser la bibliothèque d'OpenCV [1] pour réaliser les segmentations statistiques basées histogramme, en particulier le seuillage des pixels en fonction de leur intensité et la méthode d'Otsu pour le seuillage automatique, et la segmentation par classification avec la méthode de k-means.

Les fichiers de supports du TP sont à récupérer sur Arche.

Attention : Vous ne modifiez pas le fichier `main.cpp` et seulement le fichier `fonctions.cpp` est à compléter.

Dépôt sur Arche : Vous rendez le fichier `fonctions.cpp`, `main_trackbar.cpp`, `main_SLIC.cpp` et un petit rapport en pdf contenant les résultats d'exécution de votre programme.

Exercice 1 — Seuillage d'une image

Rappel : Étant donnée l'image I et un seuil s, l'image seuillée I' est calculé pour chaque point p de l'image I comme suivante :

$$I'(p) = \begin{cases} 0 \text{ si } I(p) < s \\ 1 \text{ sinon} \end{cases} \tag{1}$$

Modifiez la fonction suivante dans `fonctions.cpp` :

```
Mat seuillage(const Mat& image, int seuil) ;
```

Testez le programme sur les images : ville.png, lezard.png, toit.png avec les seuils de 50, 100, 150.

```
#Pour executer le programme compilé
./main images/ville.png 1
./main images/lezard.png 1
./main images/toit.png 1
```

Bonus : grâce au trackbar [2] d'OpenCV, vous pouvez créer une petit interface pour effectuer le seuillage avec un trackbar pour le seuil. Créez un fichier `main_trackbar.cpp` dans ce cas.

1. https://opencv.org
2. https://docs.opencv.org/4.12.0/d7/dfc/group__highgui.html#gaf78d2155d30b728fc413803745b67a9b

Exercice 2 — Seuillage hystérésis

Rappel : Étant donnée l'image I et deux seuils s_{bas} et s_{haut}, l'image seuillée I' est calculé pour chaque point p de l'image I comme suivante :

$$I'(p) = \begin{cases} 1 \text{ si } I(p) > s_{haut} \\ 1 \text{ si } s_{bas} \leqslant I(p) \leqslant s_{haut} \text{ et si p est 4-connexe avec } p' \text{ tq } I(p') > s_{haut} \\ 0 \text{ sinon} \end{cases} \quad (2)$$

Modifiez la fonction suivante dans `fonctions.cpp` :

```
Mat seuillageHysteresis(const Mat& image, int seuil_bas, int seuil_haut) ;
```

Testez le programme sur les images : ville.png, lezard.png, toit.png

Voici un exemple de résultat obtenu avec le seuillage hystérésis avec seuil_bas = 60 et seuil_ haut = 98 :



Image originale

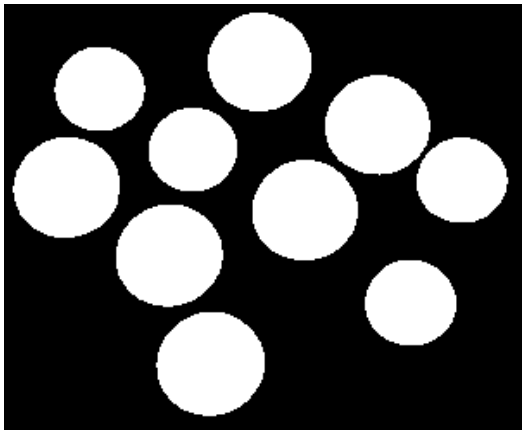


Image résultante

```
./main images/coins.png 2
```

Bonus : Grâce au trackbar d'OpenCV, vous pouvez créer une petit interface pour effectuer le seuillage avec deux trackbar pour le seuil haut et le seul bas. Modifiez le fichier `main_trackbar.cpp` dans ce cas.

Exercice 3 — Histogramme

Implémentez la fonction suivante dans `fonctions.cpp`

```
vector<int> histogramme(const Mat& image, int intervalle) ;
```

Cette fonction permet de calculer l'histogramme d'une image en niveaux gris et retourne un tableau de taille t=(256/`intervalle` + 1) dans lequel la case i contient le nombre de pixels de l'intensité de (i∗`intervalle`) à (($i+1$)∗`intervalle`−1).

Ci-après un exemple pour l'histogramme pour un intervalle = 1 et 3 :

1	5	4	4
1	2	2	1
4	1	2	2
1	4	2	5

Intervalle = 1

valeur	0	1	2	3	4	5
quantité	0	5	5	0	4	2

Intervalle = 3

valeur	0-2	3-5
quantité	10	6

Testez la fonction `histogramme` sur les images : ville.png, lezard.png, toit.png.

```
./main images/ville.png 3
```

Exercice 4 — Algorithme d'Otsu

Vous allez implémenter maintenant le seuil automatique d'Otsu avec deux fonctions suivantes dans `fonctions.cpp` :

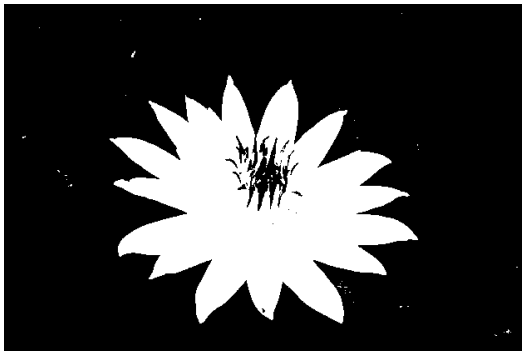
- `double calculeVariance(const vector<int>& histo, int seuil)` qui calcule la variance d'intra-classe pour un seuil donnée (cf. slides du cours)
- `int seuillageOtsu(const Mat& image)` qui calcule le seuil qui minimise la variance intra-classe en utilisant la fonction `calculeVariance`

Testez les fonctions sur les images : ville.png, lezard.png, lotus.png, toit.png et coins.png.

```
./main images/ville.png 4
```

Voici quelques résultat obtenus avec l'algorithme d'Otsu :



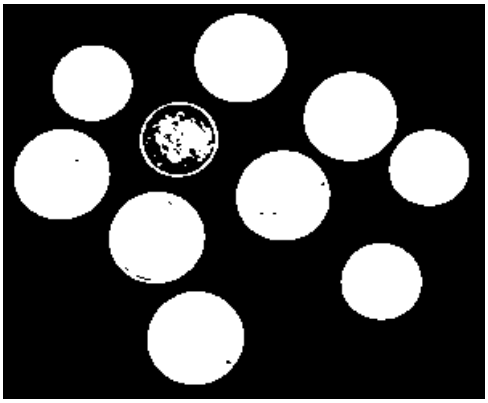








Images originales



Images résultantes

Exercice 5 — Segmentation par k-mean classification

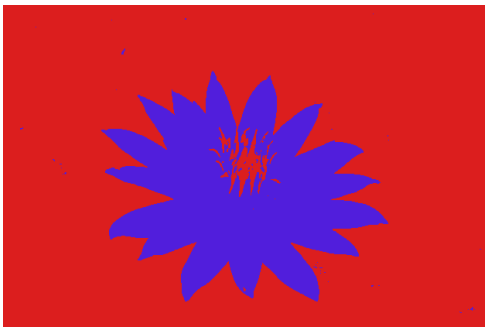
Implémentez la fonction suivante dans `fonctions.cpp` :

```
Mat kmean(const Mat& image, int k) ;
```

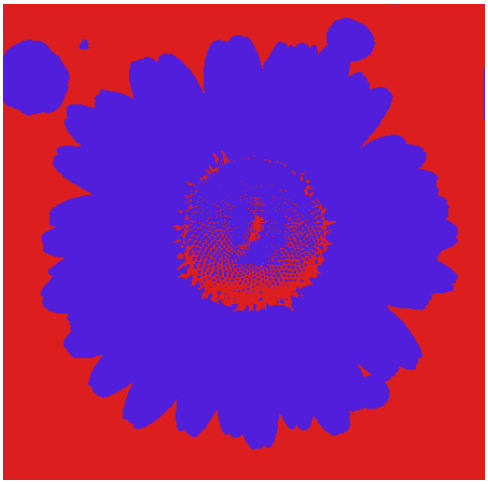
Cette fonction permet de segmenter une image par k-mean méthode, c-à-d de classer les pixels dans l'image en k classes (cf. slides du cours). Les entrées de la fonction sont : une image et le nombre de classes. La fonction retourne l'image segmentée en k classes.

Rappel : la méthode utilise l'histogramme de l'image (avec l'intervalle de 1) pour calculer itérativement les k centres dont chacun correspond à une classe. Puis utiliser ces centres pour réaliser le seuillage pour la segmentation. Voici quelques résultat obtenus avec l'algorithme k-means sur lotus.png et daisy.png avec deux classes (k=2) :









Images originales Images résultantes

Testez la fonction d'OpenCV

Exercice 6 Super-pixels avec OpenCv

Cette exercice a pour l'objectif de tester l'implémentation de l'algorithme SLIC (Simple Linear Iterative Clustering) [3] [4] dans OpenCV de La méthode proposé dans le papier : 'R. Achanta, A. Shaji, K. Smith, A. Lucchi, P.l Fua, and S. Susstrunk. Slic superpixels compared to state-of-the-art superpixel methods. IEEE PAMI 34(11) :2274–2282, 2012. Il s'agit de la fonction suivante :

```
Ptr< SuperpixelSLIC > cv::ximgproc::createSuperpixelSLIC (InputArray image,
              int algorithm=SLICO, int region_size=10, float ruler=10.0f) ;
avec
image : image entrée
algorithm : l'algorithm utilisé pour générer des super-pixels
              (SLIC=0, SLICO=1, MSLIC=2)
region_size  : la taille (en nombre de pixels) de chaque super-pixel
ruler : le facteur de lissage entre les superpixels
```

Une fois créer l'objet de super-pixels, vous devez l'itérer pour générer la segmentation avec les super-pixels avec la fonction :

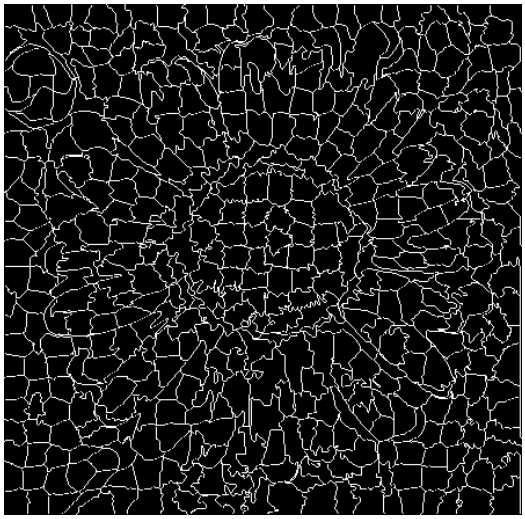
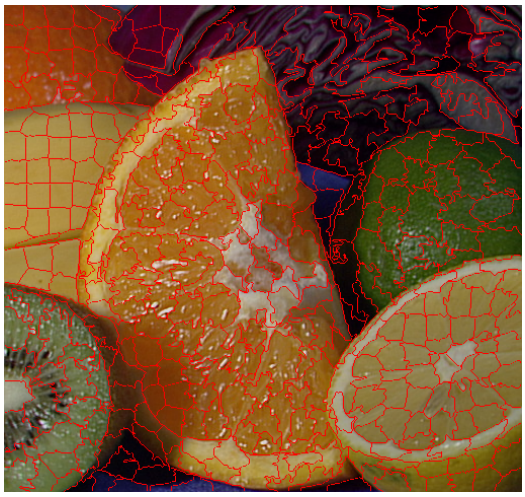
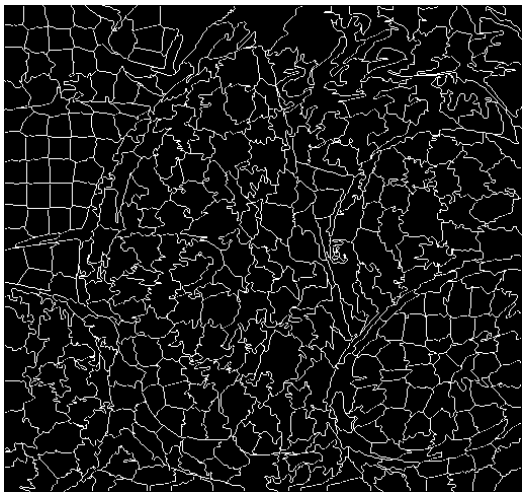
```
void iterate (int num_iterations=10);
```

Pour récupérer la masque de la segmentation par les super-pixels, vous pouvez utiliser la fonction :

```
void getLabelContourMask (OutputArray image, bool thick_line=true);
```

Créer un fichier `main_SLIC.cpp` pour tester les fonctions ci-dessus et générer la segmentation par les super-pixels pour les images : daisy.png, fruits.png avec k = 500. Sauvegardez l'image résultant avec `imwrite`. Voici quelques résultat obtenus avec l'algorithme SLIC d'OpenCV :

3. https://docs.opencv.org/4.12.0/df/d6c/group__ximgproc__superpixel.html
4. https://docs.opencv.org/4.12.0/d3/da9/classcv_1_1ximgproc_1_1SuperpixelSLIC.html

Résultats de super-pixels

Masques de super-pixels